Java Collections Framework - Interview Notes

1. Differences Between Set, List, Queue, and Map

Feature	Set	List	Queue	Map
Duplicates allowed	No	Yes	Usually No	Keys: No, Values: Yes
Ordering	No/Insertion/ Sorted	Maintains insertion	FIFO/priority	No/Insertion/Sorted
Index-based access	No	Yes	No	No (access by key)
Common implementations	HashSet, LinkedHashSet, TreeSet	ArrayList, LinkedList, Vector	LinkedList, PriorityQueue, ArrayDeque	HashMap, LinkedHashMap, TreeMap, Hashtable
Use case	Unique elements	Ordered collection	Task scheduling, event processing	Key-value lookup

2. HashSet vs LinkedHashSet vs TreeSet

Feature	HashSet	LinkedHashSet	TreeSet
Duplicates allowed	No	No	No
Order	Unordered	Insertion order	Sorted order
Performance	O(1)	Slightly slower	O(log n)
Null elements	Allowed (1)	Allowed (1)	Not allowed
Use case	Fast, unordered unique elements	Maintain insertion order	Sorted unique elements

3. ArrayList vs LinkedList vs Vector

Feature	ArrayList	LinkedList	Vector
Duplicates allowed	Yes	Yes	Yes
Null elements	Allowed	Allowed	Allowed
Insertion order	Yes	Yes	Yes

Feature	ArrayList	LinkedList	Vector
Thread-safe	No	No	Yes (synchronized)
Performance	Fast random access, slow insert/delete	Slow random access, fast insert/delete	Same as ArrayList but synchronized
Memory overhead	Less	More (next/prev links)	Same as ArrayList
Use case	Read-heavy, random access	Frequent insert/delete	Legacy, thread-safe requirement

4. HashMap vs LinkedHashMap vs Hashtable vs TreeMap

Feature	HashMap	LinkedHashMap	Hashtable	TreeMap
Duplicates allowed	Keys: No, Values: Yes	Keys: No, Values: Yes	Keys: No, Values: Yes	Keys: No, Values: Yes
Order	No	Insertion order	No	Sorted order
Thread-safe	No	No	Yes	No
Null keys/ values	1 key, multiple values	1 key, multiple values	Not allowed	Not allowed
Performance	O(1)	O(1) slightly slower	Slower due to synchronization	O(log n)
Use case	General purpose key-value mapping	Maintain insertion order	Legacy thread-safe map	Automatically sorted map

Interview-style Notes

- **List:** Use when you need ordered collection, duplicates allowed, index access.
- **Set:** Use when uniqueness is required, no duplicates.
- **Queue:** Use for FIFO or priority processing.
- **Map:** Use for key-value mapping, fast retrieval by key.
- **Thread-safety:** Vector & Hashtable are legacy; prefer synchronized collections or concurrent collections.
- **Performance tips:**
- ArrayList: fast random access
- LinkedList: frequent insert/delete
- HashMap: general-purpose key-value
- TreeSet/TreeMap: only if sorted order is required